Data Onboarding Guidelines – Azure Data Engineering

Author Name	Ashok Amsamani
Profession	Azure Data and AI Engineer
Location	Dubai
Contact	https://www.linkedin.com/in/ashok-amsamani/ https://topmate.io/ashok_amsamani

The **Data Onboarding Guidelines – Azure Data Engineering** document is a structured and practical reference for data teams tasked with **ingesting, transforming, and managing new data sources within the Azure ecosystem.** It outlines a step-by-step approach starting from **requirement gathering, assessing source systems, designing target storage layers, and defining medallion architecture (Bronze, Silver, Gold)** with proper tooling and orchestration.

It also includes detailed checklists for **Spark performance optimization, cluster selection, CI/CD deployment strategies, data governance and security policies, retention and archival management, and an incident response playbook.** The document ensures alignment with business goals, compliance standards, and operational SLAs—making it an essential resource for building scalable, secure, and efficient data pipelines on Azure.

help

Data Onboarding Guidelines – Azure Data Engineering

1. Requirement Gathering

When onboarding new data into Azure, requirement gathering is a critical phase. You need to ask comprehensive and structured questions to ensure you understand the data sources, business goals, technical requirements, compliance concerns, and end-user needs.

Here's a categorized list of essential questions to ask:

- What is the business goal or use case for onboarding this data?
  - This defines why the data is needed and what value it brings—whether it's for reporting, customer analytics, fraud detection, personalization, etc.
  - Helps align technical design with business expectations and ensures that only relevant data is ingested, avoiding wasteful storage and compute usage.
- Who are the key stakeholders (business owner, IT owner, data steward etc)?
  Identifying stakeholders ensures:
  - Accountability and escalation paths
  - Clear communication during design, development, and post-deployment phases
  - Inclusion of the right people for requirement clarifications, data sign-offs, and validation
- What are the success criteria or KPIs for this onboarding?
  Success criteria help define measurable outcomes like:
  - Timeliness of data delivery
  - Accuracy and completeness of ingested data
  - Adoption by downstream team
- Who are the end consumers of the data (Data Scientists, Analysts, Apps)?
  Understanding the consumer type affects:
  - Format and granularity of data (raw vs. aggregated)
  - Performance expectations (interactive dashboards vs. offline batch ML)
  - Access control (e.g., PII restrictions for external apps, broader access for internal analytics)
- What are all the business validation rules for the data ingestion?
  These rules ensure data is meaningful and trustworthy before it reaches the consumer.
- Are there any regulatory or compliance requirements (e.g., GDPR)? If yes, identify the fields to be anonymized.
  This ensures the pipeline complies with legal frameworks like GDPR, HIPAA, CCPA:

help

    - PII/PHI fields must be masked, encrypted, or anonymized
    - Retention policies may need to be enforced. Ignoring this can lead to legal penalties or data breaches.
- How frequent data needs to be processed - real-time or batch?
  This affects the architecture design:
    - Real-time → Event Hubs, Stream Analytics, micro-batching in Databricks
    - Batch → ADF, scheduled jobs, cost-optimized compute

  It also defines SLA commitments for delivery and alerting setup.
- Ensure we don't have the data already exist in our system. Identify this from your metadata management tool.
  It Prevents duplication of data and resource waste:
    - Reduces redundancy and maintenance effort
    - Promotes reusability of existing pipelines or datasets
    - Helps in better data governance by avoiding conflicting versions

2. Source System Understanding

Understanding the source system is the foundation of any successful data onboarding effort in Azure. It ensures seamless integration, reliable data ingestion, and long-term maintainability of pipelines.

- What is the source system (e.g., SQL, SAP, REST API, SaaS, flat file, third-party)?
  Identifying the source system type helps determine the ingestion method and appropriate tools. It also helps estimate complexity, latency, and data refresh strategies.
- Who owns or manages the source system?
  This helps establish a point of contact for access, schema clarifications, SLAs, and troubleshooting. It ensures smooth collaboration and reduces delays in case of schema changes or data issues.
- Is this a push or pull mechanism? Is it event-driven?
  Knowing the data delivery method helps decide:
    - Whether the pipeline should actively poll (pull) or wait for push or event triggers.
    - It helps us to decide event-driven architecture (Event Grid, Event Hub) for real-time ingestion or scheduled pipelines for batch processing.
- What is the frequency of the data (real-time, hourly, daily, etc.) which source team can provide?
  Frequency determines the scheduling strategy (ADF triggers, Databricks jobs) and latency expectations. High-frequency data needs more robust monitoring and auto-scaling, while daily jobs can be resource-optimized.
- What is the expected data volume (initial load and incremental)?
  It helps us to decide cluster sizing, partitioning strategy and throughput optimization.

help

- What are the varieties of data (structured, semi-structured, unstructured)?
  Variety affects the parsing logic and storage structure
  - Structured → SQL schema
  - Semi-structured → JSON/XML parsing
  - Unstructured → OCR, NLP (e.g., images, PDFs)

  Different formats require different ingestion and processing pipelines.

- What is the file/data format? (CSV, JSON, XML, Avro, Parquet, etc.)
  The format defines:
  - How to read the file (e.g., Spark reader)
  - Efficiency of storage and querying (Parquet > CSV)
  - Compression and performance factors
    Certain formats (Avro, Parquet) are schema-aware and support evolution.
- Is the data sensitive or confidential? (PII, PCI, HIPAA, etc.)
  This ensures regulatory compliance (GDPR, HIPAA, etc.). We should mask the sensitive or personal data while storing in any storage layer, it should not be in plain text.
- Is historical data available? If yes, how far back?
  Historical data is essential for:
  - Initial backfill
  - Trend analysis, forecasting, ML training
  - Planning cold vs. hot storage tiers in ADLS

  Knowing the time range helps define retention policies.

- How to identify incremental data?
  Helps build efficient delta loads using last_updated, change_flag, or CDC (Change Data Capture). This reduces data processing overhead and ensures only new or changed records are processed.
- How do we identify unique records? Is there a primary key?
  Primary keys are used to:
  - Deduplicate records
  - Perform upserts (merge operations)
  - Track change history

  Without unique identifiers, data consistency becomes a challenge.

- What is the source schema definitions (header, splitter, footer, fields), data type of every field and constraints for fields (NULL, Value Range etc).

  Understanding the schema helps:
  - Build schema validation rules
  - Define data contracts
  - Prevent type mismatches or ingestion errors

help

- What is the access credentials and connection info to pull the data?
  Credentials and connection strings are required to:
    - Establish connectivity (via ADF, Linked Services, etc.)
    - Set up secure access using Key Vault
    - Ensure the pipeline has authentication/authorization in place

3. Target System Designing

Designing the target system (it includes bronze, silver and gold layer) is crucial to ensure that the onboarded data is stored efficiently, securely, and in a format suitable for downstream use (e.g., BI, ML, APIs).

- Identify the destination storage location based on consumption nature. It could be ADLS, Azure SQL Server, CosmosDB etc.

  Choosing the right store improves query performance, reduces costs, and aligns with consumer needs.

  - Structured analytical data → Azure SQL/Synapse
  - High-volume raw/semi-structured data → ADLS
  - Low-latency NoSQL needs → CosmosDB
- Define Target Database, Table Names, Fields, Data Types, and Constraints
  - Ensures schema consistency
  - Avoids type mismatch during ingestion
  - Enables data validation and referential integrity

- Business logic applied for derived or audit fields (row_id, created_timestamp etc) in destination storage.
  - Supports data lineage, troubleshooting, and audit trail
  - Enables tracking of changes for incremental loads or CDC mechanisms

- Define data partition strategy.
  Choose high cardinal column for partitioning the data.
  - Optimizes query performance and storage layout in ADLS or Synapse
  - Reduces scan time and computational costs
  - Supports scalability and parallel processing

- Data storage mechanism for personal and sensitive data.
  - Ensures compliance with data privacy laws (GDPR, HIPAA)
  - Protects against unauthorized access or misuse
  - Enables tiered access control

- Legal and compliance checks for consuming and storing the data in our system.
    - Avoids regulatory violations and fines
    - Ensures only permissible data is stored
    - Supports internal audit and governance processes

- Prepare metadata document and get sign-off from Architecture team or Manager. Get sign-off from Architecture team or Project Manager
    - Serves as a data contract between teams
    - Aids in troubleshooting, future enhancements, and team onboarding
    - Provides clarity on field purposes and transformations

4. Data Engineering Architecture

Planning your data engineering architecture ensures scalability, performance, and cost efficiency through optimal design choices like modular layers, partitioning, and efficient formats. It enables better governance with built-in lineage, security, and compliance.

- Define medallion architecture (bronze, silver, gold layers).
  Encourages modular design with clear separation of concerns:
    - Bronze: Raw data (as-is, minimal transformation)
    - Silver: Cleaned and validated data
    - Gold: Aggregated and business-ready data

  Helps in reprocessing, auditability, and debugging.

- Include flow diagrams showing data flow between layers for History, Full Load, Ad-hoc and Incremental.
    - Diagrams help visually communicate architecture to technical and non-technical stakeholders
    - Makes it easier to explain control flow, data lineage, and pipeline behavior

- Explain each data layers: data source, validations and transformations, sink, storage format (e.g., Parquet, Delta), processing engine (e.g., Spark, Photon, ADF), catalog management (e.g., Hive, Unity Catalog).
- Mention tools used for scheduling and orchestration.
- List tools used (ADF, ADB workflow, Airflow).
- Define job frequency, dependencies, retries, and SLA.

5. Spark Performance Optimization Guidelines:

Optimizing Spark jobs is essential for efficient resource utilization, faster execution, and lower costs. This checklist helps ensure your workloads scale well under large volumes of data with minimal failures.

- Push filters early (filter before transformations) to reduce data volume at source
- Prune unnecessary columns to avoid I/O overhead and memory consumption
- Avoid Python UDFs; use Pandas UDFs or native Spark functions for better performance
- Choose efficient file formats like Parquet or Delta for better compression and read speed
- OPTIMIZE + Z-Ordering (for Delta) to improve scan performance on multidimensional filters
- Use appropriate repartitioning/coalesce/caching strategy to balance performance and memory
- Select correct join strategy: use broadcast joins for small lookups, avoid expensive shuffles
- Avoid partitioning on high-cardinality columns which cause small files and metadata issues
- Leverage adaptive query execution (AQE) for auto-tuning joins and skew handling
- Manage small files: compact them using Delta's OPTIMIZE or periodic consolidation jobs
- Enable predicate pushdown when reading from external sources (e.g., JDBC, cloud storage)
- Monitor Spark metrics: shuffle size, skewed stages, spill to disk, GC time, serialization issues
- Use Spark UI and Azure Monitor to identify bottlenecks
- Set up alerts and logs for job failures, SLA breaches, long durations, and resource exhaustion
- Cache only when reused; unpersist data after use to free up memory
- Avoid wide transformations inside loops; restructure logic for parallel execution
- Use checkpointing for long lineage DAGs in iterative computations

6. Data Governance and Security

Data governance and security ensure that data is properly classified, protected, and accessed only by authorized users. It establishes trust, supports compliance, and maintains accountability across the data lifecycle. A strong governance framework is critical to managing risks, enabling self-service, and scaling responsibly.

- Data Classification and Lineage
    - Maintain table-level and column-level metadata
      Classification labels: Public, Internal, Confidential, PII, PCI, HIPAA
    - Document business meaning, owner, and sensitivity for each field

    - Track data flow between systems and layers (source → bronze → silver → gold). Use tools like Azure Purview, Unity Catalog.

- Access Control and Authorization
    - Implement RBAC (Role-Based Access Control) at:
        - Storage (e.g., ADLS, Blob)
        - Schema, table, column, and row-level security
    - Use Azure-native tools like:
        - Azure RBAC, Unity Catalog, Purview Access Policies
        - Service Principals and Managed Identities
    - Apply attribute-based access control (ABAC) for fine-grained security where needed

- Encryption, Masking, and Compliance
    - Use encryption at rest and in transit (enabled by default in Azure Storage and SQL)
    - Apply column-level masking for PII/sensitive data using:
        - Dynamic Data Masking (DDM) in Azure SQL
        - Always Encrypted for end-to-end protection in SQL
        - Encryption Algorithm like Fernet
    - Store secrets and connection strings securely in Azure Key Vault
    - Follow data protection regulations:
        - GDPR, HIPAA, CCPA, regional laws (e.g., UAE, EU data residency)

- Audit Logging and Monitoring
    - Enable audit logs for access and modification tracking
        - Use Azure Monitor, Log Analytics, SQL Audit Logs
    - Track who accessed what data, when, and what changes were made
    - Set up alerts for unusual access patterns or breaches

7. Cluster Selection – Guidelines

Choosing the right cluster configuration is critical for ensuring performance, scalability, and cost-efficiency in Spark workloads. Proper cluster design avoids overprovisioning and accelerates job execution.

- **Match Compute Type to Workload Type**

Workload Type	Recommended Compute	Reasoning
Small data (<1 GB)	Azure Functions, ADF Mapping Data Flows	Cost-effective and lightweight, ideal for simple transformations
Medium–Large Data	Azure Databricks	Supports large-scale distributed processing and transformations
Streaming Jobs	Azure Databricks Structured Streaming	Handles real-time ingestion and processing with scalable throughput
Data Science / ML	Databricks Notebooks & ML Runtime	Built-in ML libraries, GPU support, model tracking, and AutoML

- **Use the Right Cluster Type**
    - Job Cluster:
      Ideal for automated and production workloads, launched and terminated per job. Reduces cost by not running idle between tasks. It enforces isolation and reduces shared resource contention.
    - Interactive Cluster:
      Best for notebook development, debugging, and ad-hoc analysis, offering a persistent Spark session.

- **Optimize for Cost and Performance**
    - Single-node Clusters:
      Efficient for unit tests, very small datasets, or non-distributed workloads.
    - Autoscaling Clusters:
      Automatically adjust workers based on load. Avoids over-provisioning; set sensible min and max workers.
    - Fixed-size Clusters:
      Use when workload size is predictable and stable, avoiding autoscaling startup latency.

help

Tip: Monitor autoscaling behaviour with Ganglia/Spark UI to tune properly.

- **Photon Accelerator (Databricks)**
  - Enable Photon for:
    - SQL-heavy queries on Delta tables
    - High-throughput, analytical processing
    - Improving scan, join, and aggregation speed

  - Avoid Photon for:
    - Lightweight or Python-heavy workloads
    - Exploratory notebooks where startup cost outweighs benefits

  Note: Photon improves performance significantly in SQL workloads by using vectorized C++ execution.

- **Node Type Selection (VM Family)**
  - Memory-Optimized Nodes
    - Best for ETL pipelines, Delta caching, and broadcast joins.
  - Compute-Optimized Nodes
    - Ideal for CPU-heavy transformations like parsing, hashing, or sorting.
  - GPU-Enabled Nodes
    - Use for deep learning, image processing, or TensorFlow/PyTorch workloads.

8. CI/CD and Deployment

CI/CD (Continuous Integration and Continuous Deployment) ensures faster, reliable, and repeatable delivery of data engineering code. It enables automated testing, version control, and smooth promotion from development to production environments.

- **Follow Git Branching Strategy** – main, develop, feature, release
- **Implement automated tests** – perform unit and integration tests.
- **Environment Promotion Strategy** - Define and automate promotion rules (e.g., Dev → UAT → Prod) using branches or deployment stages
- **Artifact Management -** Package notebooks or scripts as versioned artifacts (e.g., using whl, egg, .dbc bundles)
- **Data Validation Post Deployment** - Include data validation or smoke tests in post-deployment steps

9. Retention, Archival, and Purging Policy

A well-defined retention, archival, and purging policy ensures data is stored for the required duration based on its value, compliance, and business needs. It helps control storage costs, minimize legal risks, and improve data management.

- **Retention Guidelines**
  - Bronze Layer (Raw/Unvalidated Data) – *Retention: 180 days*
    - Keep data just long enough to support reprocessing and troubleshooting
    - Older data is usually noisy, incomplete, or has been cleansed in later layers
  - Silver Layer (Cleaned/Validated Data) – *Retention: Up to 1 Year*
    - Retain mid-tier, query-ready data for reusability, operational analysis, and fallback if Gold fails
    - Enables downstream re-aggregation or re-computation
  - Gold Layer (Aggregated, Business-Critical Data) – *Retention: 3+ Years*
    - Store long-term for reporting, audit trails, trend analysis, and regulatory needs
    - May need to comply with industry-specific laws (e.g., finance, healthcare, government)

  Note: Retention days will vary depending on business and legal requirements.

- **Archival Strategy**
  - Move older data (past retention) from hot to cool/archive tiers in ADLS Gen2
  - Archive old Gold layer summaries in compressed formats (e.g., Parquet, Avro)
  - Tag archived files with metadata (e.g., archived_date, retention_end)
  - Automate archiving using ADF pipelines, Databricks jobs, or Lifecycle Policies

- **Purging Strategy**
  - Purge expired data securely using:
    - ADF Delete activity
    - Databricks cleanup jobs
    - Lifecycle policies on containers/folders
  - Ensure:
    - Audit logs for deleted records
    - Purging follows regulatory compliance policies
    - No dependent data or indexes are broken by deletion

10. Incident Playbook

An incident playbook provides a structured approach to identify, respond to, and resolve failures in data pipelines and infrastructure. It ensures minimal downtime, faster recovery, and clear ownership during critical issues.

- **Document - Failure Types, Causes, and Resolution Steps**
  - Categorize incidents:
    - Pipeline failures (e.g., Spark job crash, timeout)
    - Data issues (e.g., schema mismatch, null values, missing data)
    - Infrastructure failures (e.g., cluster capacity, storage access issues)
    - Dependency failures (e.g., upstream system down, API not responding)
  - For each category, document:
    - Root causes
    - Detection methods
    - Immediate steps to mitigate
    - Permanent resolution or fix

- **Define Incident Ownership and SLAs**
  - Assign incident owners per pipeline/system (e.g., Data Engineer A, ML Ops Lead)
  - Define severity levels:
    - P1 – Business-critical failure (e.g., Gold layer unavailable)
    - P2 – High-impact, but with a workaround
    - P3 – Minor bug, doesn't block operations
  - Set resolution SLAs based on impact (e.g., P1 within 2 hours)

- **Escalation Matrix**
  - Define who to contact and in what order based on incident type
    - Primary owner → Backup → DevOps → Manager
  - Include on-call rotations and emergency contacts

- **Post-Incident Review (PIR)**
  - Conduct Root Cause Analysis (RCA) for major incidents
  - Document:
    - What went wrong
    - What worked well
    - What should be improved
  - Track action items to prevent recurrence

For any clarifications, contact me at

https://www.linkedin.com/in/ashok-amsamani/

https://topmate.io/ashok_amsamani